



Hello, Mobile World

Mobile App Development with React Native & Expo

Session 1 of 3 · 90 minutes



Today's goal

By the end of 90 minutes you will:

1. Understand **what** React Native & Expo are
2. Run a **live app on your own phone**
3. Learn just enough **JavaScript** to be dangerous
4. Build an **"About Me" profile card** — your first screen

No experience needed. We go slow. Questions are *encouraged*.

How this works 🙌

- 💻 You **write code** on your laptop (in `App.js`)
- 📱 It **runs on your phone** through **Expo Go**
- 📶 They connect over the **same Wi-Fi**
- 💾 Save the file → the app **updates instantly** (hot reload)

That feedback loop is the whole magic. We'll use it constantly.

Part 1

What is mobile development?

Native vs. cross-platform

	Native	Cross-platform
Android	Kotlin / Java	✓ one codebase
iPhone	Swift	✓ one codebase
Teams need	2 apps, 2 skills	1 app, 1 skill
Examples	—	React Native, Flutter

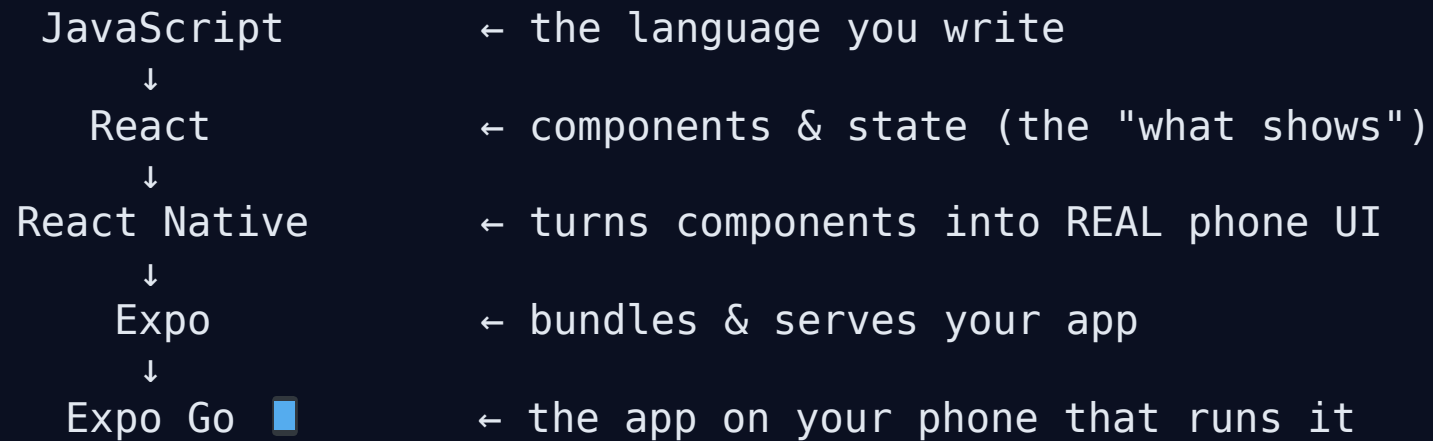
React Native lets us write **one app in JavaScript** that runs on **both** Android and iPhone.

Why React Native + Expo?

- 🌐 **One codebase** → Android **and** iPhone
- 🧰 **Expo** removes the painful setup (no Android Studio / Xcode today)
- 🔥 **Hot reload** — see changes instantly
- 🏢 Used in the real world by **Instagram, Discord, Shopify, Coinbase, Tesla**

React Native = the toolkit. **Expo** = the friendly wrapper that makes it easy to start.

The mental model 🧠



You write the **top**. Expo handles the **bottom**.

Part 2

Let's get it running

What you installed

Tool	What it's for
Node.js	Runs our developer tools
VS Code	Where you write code
Expo Go (phone)	Runs your app live

Check your laptop is ready:

```
node --version    # should print a version number
```

Create & run an app

```
# make a fresh app (blank = simplest, JavaScript)
npx create-expo-app@latest my-app --template blank

cd my-app
npx expo start          # a QR code appears
```

Then **scan the QR code**:

- **Android** → Expo Go → *Scan QR code*
- **iPhone** → Camera app → tap the banner

Hot reload = your superpower 🔥

1. Open `App.js` in VS Code
2. Change the text `"Open up App.js..."` to **your name**
3. **Save** (`Ctrl/Cmd + S`)
4. 👁️ Watch your phone update **instantly**

If it ever gets weird: press `r` in the terminal to reload, or `npx expo start -c` to clear the cache.

Part 3

A tiny bit of JavaScript

Variables — labeled boxes

```
const name = "Juan";    // can't be reassigned (use this by default)
let score = 0;          // CAN change later
score = 10;             // ✅ ok, because it's `let`
```

- `const` → a value that **won't change** (your default)
- `let` → a value that **will change**

Basic data types

```
const title    = "My Notes";    // string → text (in quotes)
const count    = 42;            // number
const isDone    = true;         // boolean → true / false
```

Three flavors of data you'll use constantly: **text**, **numbers**, **yes/no**.

Arrays — a list of things

```
const fruits = ["apple", "banana", "mango"];

fruits[0];           // "apple" (counting starts at 0!)
fruits.length;       // 3
fruits.push("kiwi") // add to the end
```

We'll store **lists of to-dos** in an array next session.

Objects — a thing with properties

```
const task = {  
  title: "Buy milk",  
  done: false,  
};  
  
task.title;    // "Buy milk"  
task.done;    // false
```

An object groups related info together. A to-do **item** is an object; a to-do **list** is an array of them.

Functions — reusable actions

```
function greet(name) {  
  return "Hello, " + name + "!";  
}  
  
greet("Maria");    // "Hello, Maria!"  
  
// modern "arrow" style (you'll see this a lot):  
const greet = (name) => "Hello, " + name + "!";
```

A function takes **inputs** and gives back an **output**.

Part 4

Your first component

Anatomy of `App.js`

```
import { Text, View } from 'react-native'; // 1. bring in pieces

export default function App() {           // 2. a component
  return (                                 // 3. describe the UI
    <View>
      <Text>Hello!</Text>
    </View>
  );
}                                           // 4. (export = "use me")
```

A **component** is a function that **returns UI**.

JSX — HTML's cousin, inside JavaScript

```
<View>  
  <Text>Hi {name}!</Text>  
</View>
```

3 rules to remember:

1. Everything returns **one parent** wrapper (`<View>`)
2. Use `{ }` to drop JavaScript in: `{name}` , `{2 + 2}`
3. Tags must **close**: `<Image />`

Part 5

Core building blocks

`<View>` — the box

The container for everything. Think of it as a `<div>` or just **a box** that holds other things and arranges them.

```
<View>  
  { /* other components go inside */ }  
</View>
```

You'll nest Views inside Views to build layouts.

<Text> — all words live here 📝

```
<Text>Hello world</Text>
```

⚠️ **The #1 beginner error:** every piece of text **must** be inside a `<Text>` .
Raw text loose in a `<View>` will crash the app.

<Image> — pictures

```
import { Image } from 'react-native';

<Image
  source={{ uri: 'https://i.pravatar.cc/300' }}
  style={{ width: 120, height: 120 }}
/>
```

A remote image **needs a width and height** or it won't show.

Putting them together

```
<View>
  <Image source={{ uri: 'https://i.pravatar.cc/300' }}
    style={{ width: 100, height: 100 }} />
  <Text>Juan dela Cruz</Text>
  <Text>Mobile Developer</Text>
</View>
```

That's already a tiny profile! Now let's make it **look good**.

Part 6

Styling & layout

The `style` prop + `StyleSheet`

```
import { StyleSheet, Text, View } from 'react-native';

<View style={styles.card}>
  <Text style={styles.name}>Juan</Text>
</View>

const styles = StyleSheet.create({
  card: { padding: 20, backgroundColor: '#EEF2FF', borderRadius: 16 },
  name: { fontSize: 24, fontWeight: 'bold', color: '#4338CA' },
});
```

Flexbox — how things arrange

Every `<View>` lays out its children with **Flexbox**.

```
<View style={{ flexDirection: 'row' }}> // side by side →  
<View style={{ flexDirection: 'column' }}> // stacked ↓ (DEFAULT)
```

⚠ On the web the default is `row`. In React Native the default is `column` (top → bottom).

Flexbox — centering

```
<View style={{  
  flex: 1,           // fill available space  
  justifyContent: 'center', // along the main axis  
  alignItems: 'center',   // across the other axis  
}}>
```

- `justifyContent` → spacing **along** the direction
- `alignItems` → alignment **across** it

Handy style properties

```
{
  padding: 16,           // space INSIDE
  margin: 12,            // space OUTSIDE
  backgroundColor: '#fff',
  borderRadius: 16,      // rounded corners
  fontSize: 18,
  fontWeight: 'bold',
  color: '#333',
}
```

Mix & match these to style anything.

Live-code target

We'll build this **Profile Card** together, then you'll make your own:

-  a round avatar image
-  a name + role
-  a short bio box
-  three info rows ( location,  school,  learning)



Activity 1

Build your "About Me" card

Time: ~25 min · **File:** `App.js`

Make the profile card **about you**:

1. Your **photo** (an `<Image>` — internet URL or `./assets`)
2. Your **name** and a **role** ("Future Developer")
3. A short **bio** in a styled box
4. **Three info rows** (📍 , 🎓 , 💡) using `flexDirection: 'row'`

Activity 1 — checkpoints & stretch

✅ **Done when:** your card shows your photo, name, bio, and 3 info rows, nicely spaced.

🌟 **Stretch goals:**

- Give the card a colored background + rounded corners + shadow (`elevation`)
- Add an emoji "skills" row
- Make the avatar a perfect circle (`borderRadius: 60` on a 120×120 image)

✓ Recap — you can now...

- Explain **React Native vs Expo** and the mental model
- Run a **live app on your phone** with hot reload
- Use core JS: **variables, arrays, objects, functions**
- Build UI with **View**, **Text**, **Image**
- Style with **StyleSheet** + **Flexbox**

That's a real foundation. 🙌

Take-home

Restyle your card & **add one new section**
(skills, hobbies, or a favorite quote).

Next session

Your card is **frozen**. Next week we make it **react to you** —
buttons, typing, and a **To-Do list** that grows. 🚀